



DIVISION OF PHYSICAL SCIENCES AND MATHEMATICS

College of Arts and Sciences | University of the Philippines Visayas

Miagao, Iloilo, 5023, Philippines | Tel. No. (033) 513-7020

Email Address: psm.upvisayas@up.edu.ph

COURSE MATERIAL

Group It, Then Run It

CMSC 124 Activity 10

Prepared by: Rene Andre Bedonia Jocsing

Published: October 08, 2026

Deadline: October 11, 2026

An operator-precedence table can tell you how an expression groups. It can't tell you what a function call does to shared state. Today you'll first build the expression tree, then run it under Java's operand-order rule, and finally test whether substitution preserves the result.

This activity is written in Java. Java's specification fixes left-to-right operand order. C and C++ leave that order to the compiler. That fixed order is the only reason today's trace has one right answer. You aren't expected to know Java beyond what's on the page.

The language changes from one activity to the next on purpose. The concept is the constant. Recognizing it across three notations lets you carry it into a fourth language nobody taught you. Laboratory Activity 3 is where these runtime questions come back as code, since that activity asks you to decide what your own language does with runtime types and truthiness.

You have 60 minutes for the three checkpoints. Keep each worked example beside the checkpoint it supports. Discussion with classmates and the instructor is allowed and encouraged throughout. This activity is worth 12 points. Six points reward constructions, and six reward explanations.

The Java Rules

Use these rules and no guessed rearrangements:

1. Multiplication has higher **precedence** than addition and subtraction, so it groups first.
2. Addition and subtraction have equal precedence and are **left associative**, so an unparenthesized chain groups from the left.
3. Java evaluates the left operand of a binary operator before its right operand.
4. A **side effect** changes state beyond producing the expression's value.

Precedence and associativity determine the **operator tree**, the hierarchy of operator nodes and operand children. **Operand evaluation order** determines which subtree runs first. A **side effect** changes program state in addition to producing a value. An expression is **referentially transparent** when replacing it with its value preserves program behavior.

Worked Example for Checkpoint 1

Suggested time: 18 minutes, including the worked example.

```
1 static int n = 4;
2
3 static int change() {
4     n = 9;
5     return 2;
6 }
```

```

7
8  int answer = n + change() * 2 - n;

```

`change()` is a bad function on purpose. It writes to a global and hands back a number unrelated to what it wrote. Nobody should ship that. It's here because a function doing both is what makes operand order visible on the page. Programs that do it by accident are the reason the rule exists. `shift()` in the checkpoint is written the same way.

The initializer groups as $(n + (change() * 2)) - n$.

Read the grouping as “the left n plus the result of $change$ times two, then subtract the right n .”

Multiplication sits lower in the tree than addition, while left associativity makes subtraction the root. These rules build the tree before any operand runs. Making multiplication the root because it has the highest precedence reverses the tree levels. Higher precedence places an operator lower, closer to its operands. Read the root as the operator that runs last, since it can't combine anything until both subtrees are finished. Binding tightest therefore means sitting deepest.

The three figures below share one tree. The tree never changes. The values read off its leaves do because a call sits between the two reads of `n`.

Group first, then let state change

The tree never changes. The values read from its leaves can change while Java evaluates it from left to right.

Build the operator tree

Multiplication sits below addition, and left associativity makes subtraction the root.

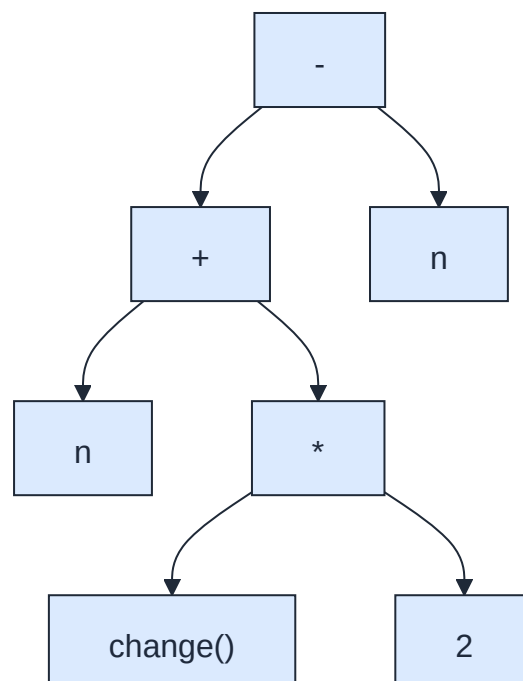


Figure 1: Build the operator tree

Evaluate through the side effect

The left `n` contributes 4. `change()` then sets `n` to 9 and returns 2, so its multiplication contributes 4.

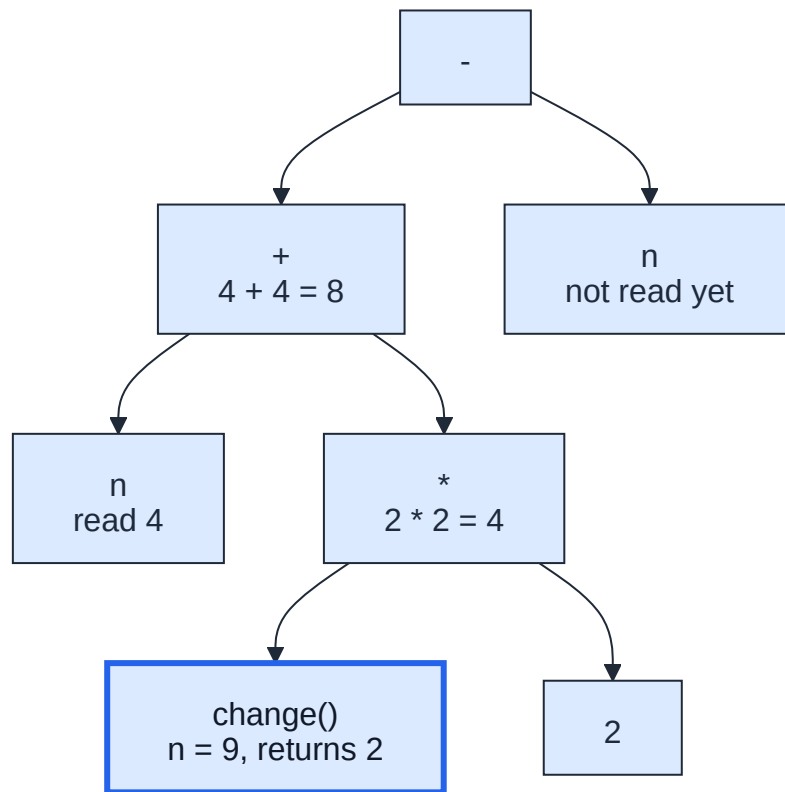


Figure 2: Evaluate through the side effect

Read the changed state

The right `n` now contributes 9, so the complete result is $8 - 9 = -1$.

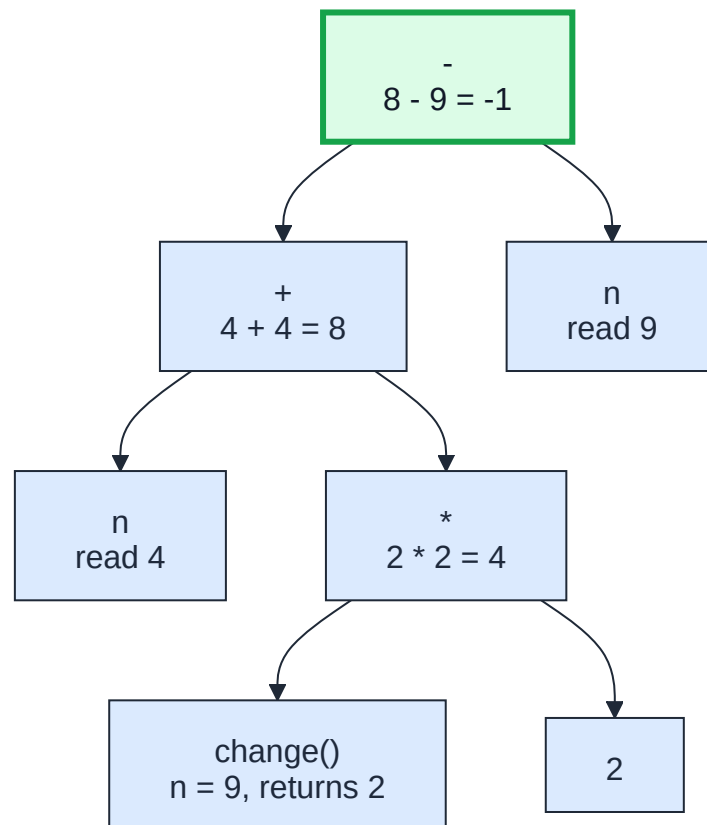


Figure 3: Read the changed state

Checkpoint 1: Group Before Running (4 Points)

Use Java's grouping rules: multiplication binds more tightly than addition and subtraction, while addition and subtraction have equal precedence and associate left. Apply them to this initializer before evaluating any operand:

```

1  static int meter = 7;
2
3  static int shift() {
4      meter = 12;
5      return 3;
6  }
7
8  int result = meter + shift() * 2 - meter;

```

1. (Construction: 2 points) Fully parenthesize the initializer for `result`, then draw its operator tree.

2. (Explanation: 2 points) Answer both parts.

- Label the part of the grouping that precedence decided and the part that associativity decided.
- Explain why neither label tells you what value either `meter` leaf will read.

Run the Tree

Worked Example for Checkpoint 2

Suggested time: 24 minutes, including the worked example.

```

1  static int n = 4;
2
3  static int change() {
4      n = 9;
5      return 2;
6  }
7
8  int answer = n + change() * 2 - n;

```

Java evaluates the worked expression above, `answer`, in this order:

Step	Value	State of <code>n</code>
read the left <code>n</code>	4	4
call <code>change()</code>	2	9
multiply by 2	4	9
add to the saved left value	8	9
read the right <code>n</code>	9	9
subtract $8 - 9 = -1$		9

Replacing `change()` with its returned value 2 removes the side effect. The changed expression reads 4 at both `n` leaves and produces $(4 + 4) - 4 = 4$, leaving `n` = 4. The governing rule is referential transparency. Substitution is safe only when evaluating the replaced expression has no behavior that the replacement removes. This would mean that the replaced expression had no side effect, and it was referentially transparent.

Reading `n` once and reusing 4 at both leaves ignores the tree's two evaluations of `n`, which have a state-changing call between them.

Java's left-to-right rule does the work here. Suppose a language evaluated right operands first instead. Then `n` on the right gets read before the call, giving 4, while `n` on the left gets read after it, giving 9. That flips the result of the same tree to $13 - 4 = 9$. C and C++ leave this choice to the compiler, so the expression has no single answer there.

Checkpoint 2: Run the Tree (5 Points)

Running Case

```

1  static int meter = 7;
2
3  static int shift() {
4      meter = 12;
5      return 3;
6  }
7
8  int result = meter + shift() * 2 - meter;

```

The two reads of `meter` happen at different moments in the evaluation order. The call to `shift` sits between them.

1. (Construction: 3 points) Trace Java's evaluation in order. Record both reads of `meter`, the call's returned value, the state change inside `shift`, each arithmetic result, and the final values of `result` and `meter`.

2. (Explanation: 2 points) Explain why reading `meter` twice isn't equivalent to remembering its first value. Identify the side effect between the reads.

Transfer by Substitution

Worked Example for Checkpoint 3

Suggested time: 18 minutes, including the worked example.

```

1  static int n = 4;
2
3  static int change() {
4      n = 9;
5      return 2;
6  }
7
8  int answer = n + change() * 2 - n;
```

In the worked expression, `answer`, replacing `change()` with `2` removes the assignment `n = 9`. The changed expression reads `4` at both `n` leaves, produces `4`, and leaves `n = 4`. The original produces `-1` and leaves `n = 9`.

Version	Expression result	Final <code>n</code>
call <code>change()</code>	<code>-1</code>	<code>9</code>
replace call with <code>2</code>	<code>4</code>	<code>4</code>

Equal returned values don't make the two expressions interchangeable when one evaluation changes shared state.

Checkpoint 3: Test Transparency (3 Points)

```

1  static int meter = 7;
2
3  static int shift() {
4      meter = 12;
5      return 3;
6  }
7
8  // Original initializer:
9  int result = meter + shift() * 2 - meter;
10
11 // Changed initializer:
12 int result = meter + 3 * 2 - meter;
```

Compare the original initializer `meter + shift() * 2 - meter` with the changed initializer `meter + 3 * 2 - meter`, each in their own runs of the program. Start each run from `meter = 7`. The first version calls `shift()`, which sets `meter = 12`. The second version makes no call.

1. (Construction: 1 point) Evaluate the changed initializer and record the final `result` and `meter`.

2. (Explanation: 2 points) Compare `meter + 3 * 2 - meter` with `meter + shift() * 2 - meter`. Explain why the substitution fails even though `shift()` returned `3`, and state the condition under which that substitution would've been safe. End with the two final states side by side and the side-effect rule that accounts for their difference.